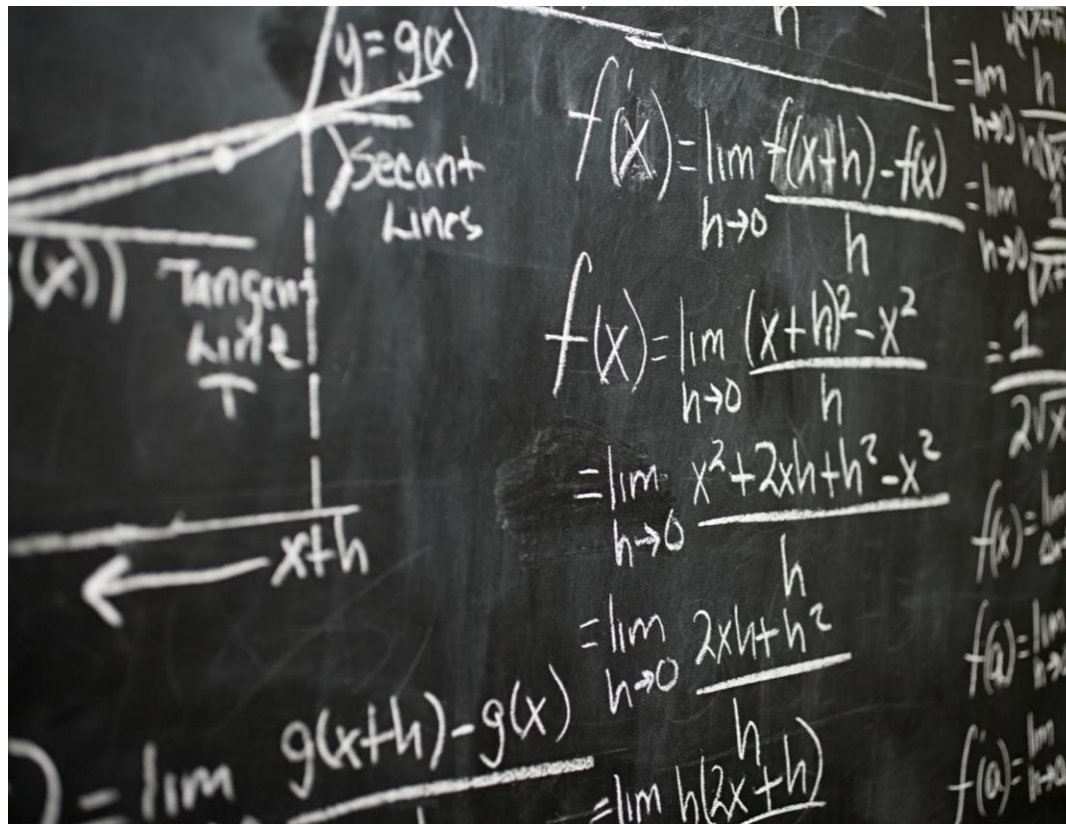




- La aritmética es un tema bastante extenso en el lenguaje ensamblador, por lo que lo dividiremos en pasos. Aquí nos enfocaremos en la suma y resta de enteros.
- Vamos a empezar con INC (incremento), DEC (decremento), ADD, SUB y NEG (negación).



- Las instrucciones INC (incremento) y DEC (decremento) suman 1 y restan 1 de un solo operando, respectivamente.
- La sintaxis es:

```
INC  reg/mem  
DEC  reg/mem
```



```
.data
miPalabra WORD 1000h
.code
inc     miPalabra                ; 1001h
mov     bx,miPalabra
dec     bx                      ; 1000h
```

Las banderas Desbordamiento, Signo, Cero, Acarreo auxiliar y Paridad cambian de acuerdo al valor del operando de destino. No afectan a la bandera Acarreo (lo cual es un poco sorprendente).



- La instrucción ADD suma un operando de origen con uno de destino del mismo tamaño. La sintaxis es:

ADD dest,origen

- *Origen* permanece sin cambio en la operación, y la suma se almacena en el operando de destino. El conjunto de posibles operandos es el mismo que para la instrucción MOV.

INSTRUCCIÓN ADD



```
.data
var1 DWORD 10000h
var2 DWORD 20000h
.code
mov    eax,var1           ; EAX = 10000h
add    eax,var2           ; EAX = 30000h
```

**HE AQUÍ UN EJEMPLO BREVE QUE SUMA DOS ENTEROS DE
32 BITS:**

INSTRUCCIÓN SUB

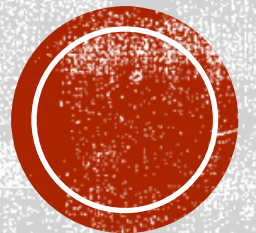
- La instrucción SUB resta un operando de origen a un operando de destino. El conjunto de posibles operaciones es el mismo que para las instrucciones ADD y MOV .
- La sintaxis es:

SUB *dest, origen*



```
.data
var1 DWORD 30000h
var2 DWORD 10000h
.code
mov    eax,var1           ; EAX = 30000h
sub    eax,var2           ; EAX = 20000h
```

**HE AQUÍ UN EJEMPLO BREVE DE CÓDIGO
QUE RESTA DOS ENTEROS DE 32 BITS:**



Acarreo: 1 1 1 1 1 1

	0	0	0	0	0	1	0	0	(4)
+	1	1	1	1	1	1	1	1	(-1)
	0	0	0	0	0	0	1	1	(3)

Una manera sencilla de realizar una resta sin tener que crear nuevos circuitos digitales es negar y después sumar. Por ejemplo, $4 - 1$ puede interpretarse como $4 - (1)$. Para los números negativos se utiliza la notación de complementos a dos, por lo que 1 se representa mediante 11111111:



INSTRUCCIÓN NEG

- La instrucción NEG (negación) invierte el signo de un número, convirtiéndolo en su complemento a dos. Se permiten los siguientes operandos:

NEG *reg*

NEG *mem*

(Recuerde que para encontrar el complemento a dos de un número se invierten todos los bits en el operando de destino y se le suma 1).



IMPLEMENTACIÓN DE EXPRESIONES ARITMÉTICAS

- Armado con las instrucciones ADD, SUB y NEG, tiene los medios para implementar expresiones aritméticas que involucran la suma, resta y negación en lenguaje ensamblador. En otras palabras, uno puede simular lo que un compilador en C++ podría hacer al leer una expresión tal como:

`valR = -valX + (valY - valZ);`

- Se utilizarán las siguientes variables de 32 bits:

```
valR SDWORD ?  
valX SDWORD 26  
valY SDWORD 30  
valZ SDWORD 40
```



- Al traducir una expresión, evalúe cada término por separado y combine los términos al final. Primero, negamos una copia de **valX**:

```
; primer término: -valX  
mov  eax,valX  
neg  eax                                ; EAX = -26
```

- Después, **valY** se copia a un registro y se resta **valZ**:

```
; segundo término: (valY - valZ)  
mov  ebx,valY  
sub  ebx,valZ                          ; EBX = -10
```



- Por último, se suman los dos términos (en EAX y EBX):

```
; se suman los términos y se almacenan:  
add  eax,ebx  
mov  valR,eax                ; -36
```



BANDERAS AFECTADAS POR LA SUMA Y LA RESTA

- Al ejecutar instrucciones aritméticas, a menudo es conveniente saber algo acerca del resultado.

¿Es negativo, positivo o cero?

**¿Es demasiado grande o demasiado pequeño para
caber en el operando de destino?**

- Las respuestas a tales preguntas nos pueden ayudar a detectar errores de cálculo que de otra manera podrían ocasionar un comportamiento errático.





Las banderas de estado de la CPU son utilizadas para verificar los resultados de operaciones aritméticas y para habilitar instrucciones de bifurcación condicional, elementos fundamentales en la lógica de programación. A continuación, se presenta un resumen de estas banderas, que se explorarán con mayor detalle posteriormente:



1. **Bandera de Acarreo:** Indica un desbordamiento en operaciones de enteros sin signo. Por ejemplo, se activa si una operación con un destino de 8 bits produce un resultado mayor que 11111111 en binario.
2. **Bandera de Desbordamiento:** Señala un desbordamiento en operaciones de enteros con signo. Por ejemplo, se activa si una operación con un destino de 16 bits produce un resultado negativo menor que -32768 en decimal.
3. **Bandera de Cero:** Indica que una operación produce un resultado igual a cero. Por ejemplo, se activa cuando se resta un operando de igual valor.
4. **Bandera de Signo:** Se activa cuando una operación genera un resultado negativo. Esto sucede al activar el bit más significativo del operando de destino.
5. **Bandera de Paridad:** Cuenta el número de bits con valor 1 en el byte menos significativo del operando de destino.
6. **Bandera Acarreo Auxiliar:** se activa cuando un bit 1 se acarrea hacia fuera de la posición 3 en el byte menos significativo del operando destino.




```
mov    ecx,1
sub    ecx,1    ; ECX = 0, ZF = 1
mov    eax,0FFFFFFFFh
inc    eax                      ; EAX = 0, ZF = 1
inc    eax                      ; EAX = 1, ZF = 0
dec    eax                      ; EAX = 0, ZF = 1
```

OPERACIONES SIN SIGNO: CERO, ACARREO Y ACARREO AUXILIAR.

La bandera Cero se activa cuando el resultado de una operación aritmética es cero. Los siguientes ejemplos muestran el estado del registro de destino y de la bandera Cero, después de ejecutar las instrucciones SUB, INC y DEC:



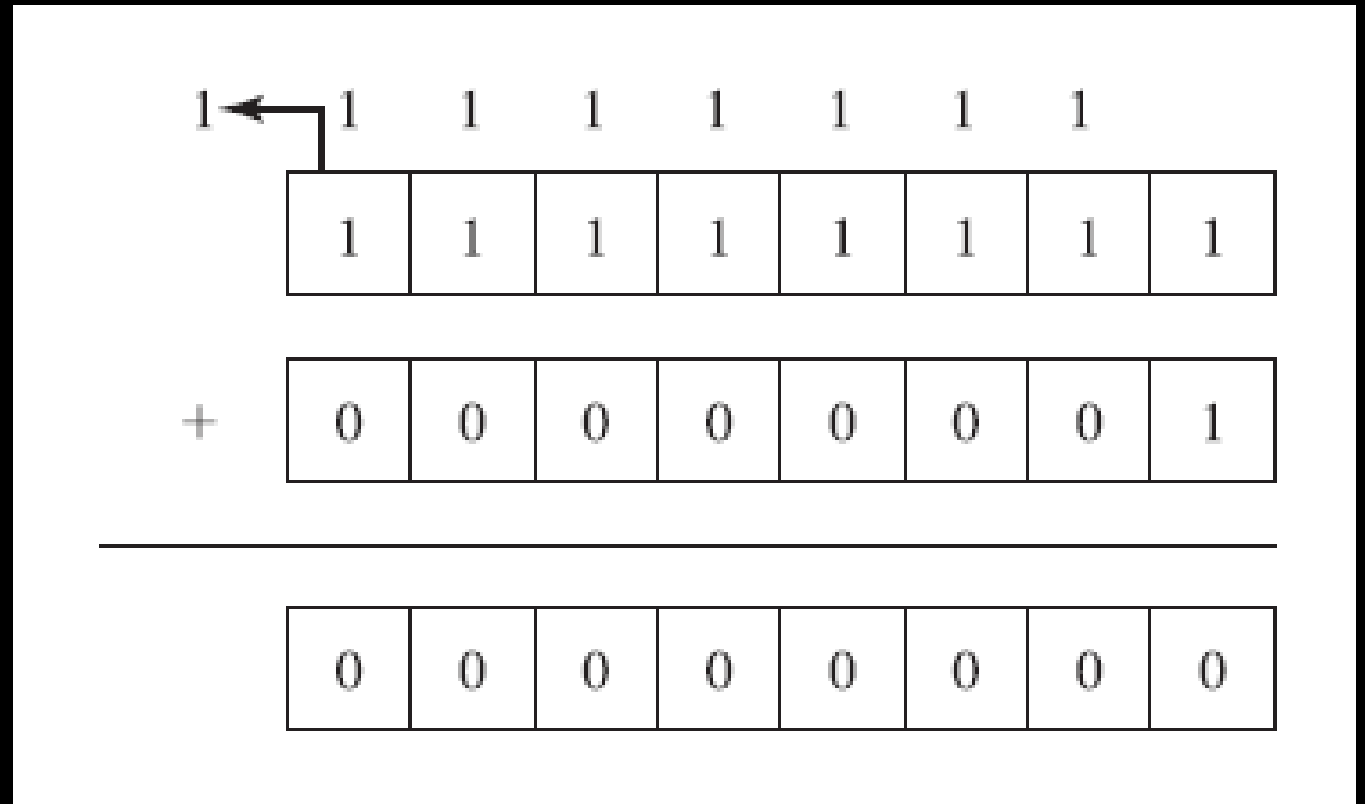
La operación de la bandera de Acarreo es más fácil de entender si dividimos la suma y la resta en casos separados. En el contexto de la suma de dos números enteros sin signo, la bandera de Acarreo refleja el acarreo que proviene del bit más significativo (MSB) del operando de destino. En otras palabras, la bandera de Acarreo (CF) se establece en 1 cuando la suma excede la capacidad de almacenamiento del operando de destino. En el siguiente ejemplo, la instrucción ADD activa la bandera de Acarreo debido a que la suma (100h) es demasiado grande para el registro AL.

```
mov  al,0FFh  
add  al,1
```

```
; AL = 00, CF = 1
```



La siguiente figura muestra lo que ocurre a nivel de bits cuando se suma un 1 a 0FFh. El acarreo que sale de la posición del bit más alto de AL se copia en la bandera Acarreo:



- Por otro lado, si se suma 1 a 00FFh en AX, la suma cabe fácilmente en 16 bits y la bandera Acarreo se borra:

```
mov  ax,00FFh  
add  ax,1                                ; AX = 0100h, CF = 0
```

- Pero si se suma 1 a FFFFh en el registro AX, se genera un Acarreo hacia fuera de la posición del bit superior de AX:

```
mov  ax,0FFFFh  
add  ax,1                                ; AX = 0000, CF = 1
```



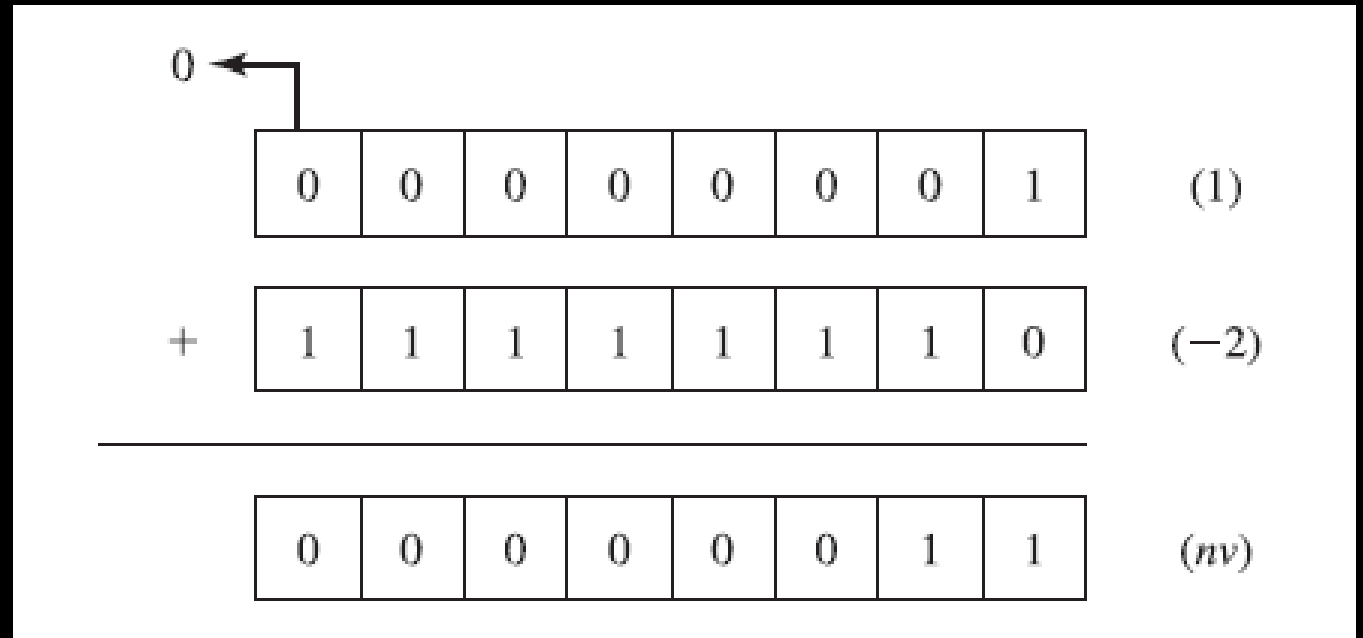


LA RESTA Y LA BANDERA ACARREO

- Una operación de resta activa la bandera Acarreo cuando se resta un entero sin signo más grande de uno más pequeño. Es más fácil considerar el efecto de la resta sobre la bandera Acarreo desde un punto de vista relacionado con el hardware. Vamos a suponer por un momento que la CPU puede negar un entero positivo sin signo, formando su complemento a dos:
 1. El operando de origen se niega y se suma al destino.
 2. El acarreo que sale del MSB se invierte y se copia a la bandera Acarreo.



Vamos a restar
2 de 1, como
operandos de 8
bits. Después
de negar el 2,
sumamos los
enteros:



Las instrucciones INC y DEC no afectan a la bandera Acarreo. Si se aplica NEG a un operando distinto de cero, siempre se activa la bandera Acarreo.

```
mov  al,0Fh
add  al,1
```

; AC = 1

He aquí la aritmética:

$$\begin{array}{r} 0\ 0\ 0\ 0\ 1\ 1\ 1\ 1 \\ +\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1 \\ \hline 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0 \end{array}$$

ACARREO AUXILIAR

- La bandera Acarreo auxiliar (AC) indica un acarreo o préstamo (borrow) en el bit 3 del operando de destino. Se utiliza principalmente en la aritmética con números decimales codificados en binario (BCD), pero puede usarse en otros contextos. Suponga que sumamos 1 a 0Fh. La suma (10h) contiene un 1 en la posición del bit 4 que se acarreó de la posición del bit 3:



- *Paridad* La bandera Paridad (PF) se activa cuando el byte menos significativo del destino tiene un número par de bits que son 1. Las siguientes instrucciones ADD y SUB alteran la paridad de AL:

```
mov  al,10001100b
add  al,00000010b      ; AL = 10001110, PF = 1
sub  al,10000000b      ; AL = 00001110, PF = 0
```

- Después de la instrucción ADD, AL contiene el número binario 10001110 (cuatro bits 0 y cuatro bits 1), y PF = 1. Después de SUB, AL contiene un número impar de bits 1, por lo que PF = 0.



- *Bandera Signo* La bandera Signo se activa cuando el resultado de una operación aritmética con signo es negativo. El siguiente ejemplo resta un entero más grande (5) de un entero más pequeño (4):

```
mov  eax,4  
sub  eax,5                ; EAX = -1, SF = 1
```

- Desde un punto de vista mecánico, la bandera Signo es una copia del bit superior del operando de destino. El siguiente ejemplo muestra los valores hexadecimales de BL cuando se genera un resultado negativo:

```
mov  bl,1                 ; BL = 01h  
sub  bl,2                 ; BL = FFh (-1)
```

OPERACIONES CON SIGNO: BANDERA SIGNO Y DESBORDAMIENTO



- **Bandera Desbordamiento** La bandera Desbordamiento se activa cuando el resultado de una operación aritmética con signo provoca que el operando de destino tenga un desbordamiento por exceso (overflow) o por defecto (underflow). Por ejemplo, del capítulo 1 sabemos que el valor de byte entero con signo más grande posible es 127; si se le suma 1 se produce un desbordamiento por exceso (overflow):

```
mov  al, +127
add  al, 1                                ; OF = 1
```

- De manera similar, el valor de byte entero con signo más pequeño posible es -128. Si le restamos 1, se produce un desbordamiento por defecto (underflow). El valor del operando de destino no almacena un resultado aritmético válido, por lo que se activa la bandera Desbordamiento:

```
mov  al, -128
sub  al, 1                                ; OF = 1
```





- *La prueba de suma* Hay una manera muy sencilla de saber si ha ocurrido un desbordamiento con signo cuando se suman dos operandos. El desbordamiento ocurre cuando:
 - Dos operandos positivos generan una suma negativa.
 - Dos operandos negativos generan una suma positiva.
- El desbordamiento nunca se produce cuando los signos de los dos operandos de la suma son distintos.



- *Cómo el hardware detecta el desbordamiento*
- La CPU utiliza un interesante mecanismo para determinar el estado de la bandera Desbordamiento, después de una operación de suma o de resta. Al bit que se acarrea hacia fuera del MSB (bit más significativo) de un operando se le aplica un OR exclusivo con el bit que se acarrea hacia el MSB. El valor resultante se coloca en la bandera Desbordamiento. Por ejemplo, al sumar los enteros binarios de 8 bits 10000000 y 11111110 no se produce ningún acarreo hacia el bit 7 (MSB), pero sí hay un acarreo del bit 7 a la bandera Acarreo:

		No hay acarreo del bit 6 al 7						
		7	6	5				
CF = 1 ←		1	0	0	0	0	0	0
	+	1	1	1	1	1	1	0
	=	0	1	1	1	1	1	0

En otras palabras, la operación 1 XOR 0 produce OF 1.



- *Instrucción NEG* La instrucción NEG produce un resultado inválido si el operando de destino no puede almacenarse en forma correcta. Por ejemplo, si movemos 128 a AL y tratamos de negarlo, el valor correcto (128) no cabrá en AL. La bandera Desbordamiento se activa, indicando que AL contiene un valor inválido:

mov al,-128 ; AL = 10000000b

neg al ; AL = 10000000b, OF = 1

- Por otro lado, si 127 se niega, el resultado es válido y la bandera Desbordamiento se borra:

mov al,+127 ; AL = 01111111b

neg al ; AL = 10000001b, OF = 0



PROGRAMA EJEMPLO

```
; ADD, SUB, INC, DEC y NEG, y la manera en
; que afectan a las banderas de estado de la CPU.
; Última actualización: 06/01/2006
INCLUDE Irvine32.inc
.data
valR    SDWORD ?
valX    SDWORD 26
valY    SDWORD 30
valZ    SDWORD 40
.code
main PROC
    ; INC y DEC
    mov     ax,1000h
    inc     ax                ; 1001h
    dec     ax                ; 1000h
    ; Expresión: valR = -valX + (valY - valZ)
    mov     eax,valX
    neg     eax                ; -26
    mov     ebx,valY
    sub     ebx,valZ          ; -10
    add     eax,ebx
    mov     valR,eax          ; -36
    ; Ejemplo de la bandera Cero:
    mov     cx,1
    sub     cx,1                ; ZF = 1
    mov     ax,0FFFFh
    inc     ax                ; ZF = 1
    ; Ejemplo de la bandera Signo:
    mov     cx,0
    sub     cx,1                ; SF = 1
    mov     ax,7FFFh
    add     ax,2                ; SF = 1
    ; Ejemplo de la bandera Acarreo:
    mov     al,0FFh
    add     al,1                ; CF = 1,  AL = 00
    ; Ejemplo de la bandera Desbordamiento:
    mov     al,+127
    add     al,1                ; OF = 1
    mov     al,-128
    sub     al,1                ; OF = 1
    exit
main ENDP
END main
```

CUESTIONARIO

